

SYNTHFIX: Adaptive Neuro-Symbolic Code Vulnerability Repair

Yifan Zhang¹ Jieyu Li¹ Kexin Pei² Yu Huang¹ Kevin Leach¹

Vanderbilt University¹ University of Chicago²
{yifan.zhang, 2, yu.huang, kevin.leach}@vanderbilt.edu
kpei@cs.uchicago.edu

Abstract

Large Language Models (LLMs) can generate plausible code patches, but plausibility is not enough for automated repair: a patch must compile, pass tests, and remove the target vulnerability. We present SYNTHFIX, a neuro-symbolic repair framework that combines supervised repair learning with compiler-informed feedback. During training, a lightweight router selects between Supervised Fine-Tuning (SFT) for common repair patterns and Reward Fine-Tuning (RFT) for examples that benefit from symbolic feedback. The reward combines static structure, lint/compile checks, security scanning, and public execution tests where available; at inference time, the same evidence guides best-of- K candidate selection under a greedy floor. Across five code LLMs (1.3B–7B) on pyrepair, CodeFlaws, and SVEN, SYNTHFIX improves deployable repair metrics over SFT-only and RFT-only baselines, with relative gains up to 54% in functional correctness and 14% in security clearance. Our code and data are available at <https://github.com/CoderDoge1108/SynthFix>¹.

1 Introduction

Large Language Models (LLMs) can generate and edit source code with impressive fluency (Chen et al., 2021; Allamanis et al., 2018). For automated vulnerability repair, however, fluent code is not enough: a patch must compile, preserve intended behavior, pass available tests, and remove the target vulnerability (Fu et al., 2022). Neural models often optimize for textual plausibility and local syntactic patterns, yielding patches that look reasonable but fail functionally or leave the vulnerability intact (Berabi et al., 2021; Jiang et al., 2023). This motivates repair systems that combine neu-

ral generation with compiler- and analyzer-derived evidence (Shi et al., 2020; Wu et al., 2022).

The standard adaptation strategy is Supervised Fine-Tuning (SFT) on bug-fix pairs (Dong et al., 2023). SFT is efficient for common repair idioms, but it provides limited pressure toward executable or security-cleared behavior beyond the demonstrations. Reward Fine-Tuning (RFT) offers a complementary path: models can optimize against public tests, security scans, or other task feedback (Le et al., 2022; Islam and Najafirad, 2024; Fang et al., 2025; Shojaei et al., 2023). In practice, applying RFT uniformly can be costly and unstable, and our experiments show that a fixed RL schedule alone does not reliably improve deployable repairs over a strong SFT baseline.

We present SYNTHFIX, an adaptive neuro-symbolic repair framework that improves on a fixed fine-tuning recipe in two ways. First, it treats SFT and RFT as complementary repair modes: common edits are learned through direct imitation, while harder examples receive reward-driven updates. Second, it keeps symbolic evidence active at inference time, where sampled candidates are ranked by compiler-, analyzer-, and test-derived signals under a greedy floor. Symbolic feedback therefore serves both as a training signal and as the decision rule for selecting the final patch.

The framework constructs its procedural reward from complementary signals, including AST/CFG/DFG structure, linting and compile checks, security scanning, and public execution tests when available. These signals connect neural code generation to the kinds of evidence used by program-analysis tools and neuro-symbolic repair systems (Parisotto et al., 2016). A lightweight neural router then controls training. Rather than committing every batch to the same objective (Shen and Chen, 2020; de Fitero-Dominguez et al., 2024), the router selects the SFT or RFT pathway using compiler-derived features such as code length, AST

¹Our artifacts are publicly available under the MIT License.

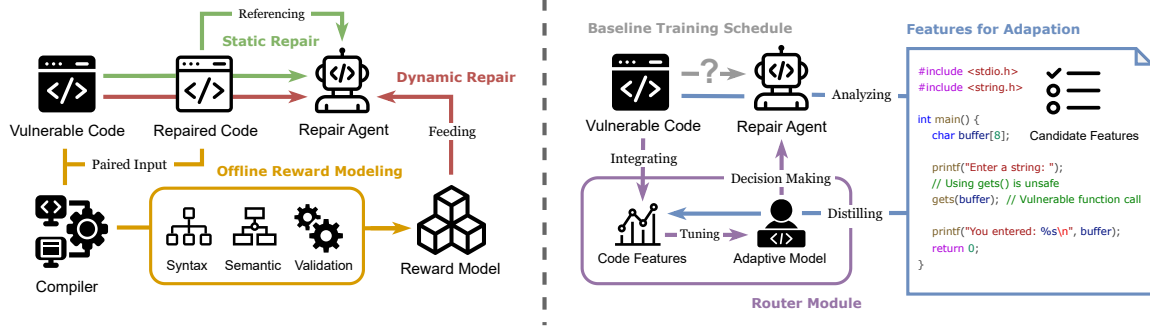


Figure 1: Overview of SYNTHFIX. The repair agent generates candidate patches, the symbolic reward model provides compiler-, analyzer-, and test-derived feedback, and the router adaptively chooses between SFT and RFT during training.

complexity, and CFG depth. This design keeps the fluent SFT prior as a stabilizing anchor while exposing harder examples to reward-driven refinement (Kulsum et al., 2024). At inference time, SYNTHFIX samples a small candidate set and applies symbolic best-of- K selection with a greedy floor, ensuring that exploration does not regress below the greedy patch.

We evaluate SYNTHFIX on three execution- and security-grounded benchmarks: pyrepair (Python, built from MBPP (Austin et al., 2021) and QuixBugs (Lin et al., 2017)), CodeFlaws for C (Tan et al., 2017), and SVEN for security repair (He and Vechev, 2023). Across five modern code LLMs (1.3B–7B), including DeepSeek-Coder (Guo et al., 2024), Llama-3.2 (Dubey et al., 2024), Qwen3 (Yang et al., 2025), CodeLLaMA (Roziere et al., 2023), and StarCoder2 (Lozhkov et al., 2024), SYNTHFIX improves deployable repair metrics over SFT-only and RFT-only baselines. The gains reach 54% relative functional improvement on CodeFlaws and 14% relative security-clearance improvement on SVEN, and module analyses show that both router-gated training and symbolic test-time selection contribute.

2 Approach

SYNTHFIX is a neuro-symbolic training and inference framework for code repair. Its central idea is to use symbolic evidence twice: during training, to decide when feedback-driven RFT should replace direct SFT; and at inference time, to choose the final patch from sampled candidates. The framework trains one **Repair Agent** and adds two lightweight components: a symbolic **Reward Model** that scores patches using compiler-, analyzer-, and test-

derived evidence, and a neural **Router Model** that selects SFT or RFT for each batch. Figure 1 gives the architecture.

2.1 Problem Formulation and Notation

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ denote a repair dataset, where x_i is a buggy program or function and y_i is the reference repair when one is available. The Repair Agent is a conditional code model $\pi_\theta(y | x)$ parameterized by LoRA-adapted weights θ . Given x , the model can produce a greedy patch $\hat{y}^{(0)}$ or a sampled set of candidate patches $\mathcal{C}_K(x) = \{\hat{y}^{(0)}, \hat{y}^{(1)}, \dots, \hat{y}^{(K-1)}\}$.

For any candidate $\hat{y}$, we compute a symbolic evidence vector $s(x, \hat{y}) = [s_{\text{parse}}, s_{\text{static}}, s_{\text{struct}}, s_{\text{sec}}, s_{\text{exec}}, s_{\text{repair}}] \in [0, 1]^m$, whose components summarize parseability, static validity, structural similarity, security-scanner feedback, public-test execution, and repair-directed edit evidence. Held-out tests are never included in s ; they are used only for final evaluation. SYNTHFIX uses this evidence vector in two places: the reward $r(x, \hat{y})$ used for RFT updates and the selector score $q(x, \hat{y})$ used to choose the final patch at inference time.

2.2 Symbolic Reward Model for Patch Evaluation

The Reward Model assigns each candidate patch a procedural score rather than relying on a single binary pass/fail outcome: $r(x, \hat{y}) = \lambda^\top s(x, \hat{y}) = \sum_{c=1}^m \lambda_c s_c(x, \hat{y})$, where λ_c controls the contribution of each symbolic component. This reward is deterministic and reference-free except for optional structural comparison terms used when a reference repair is available. It gives RFT feedback about syntax, structure, security, and executable behavior without consulting held-out tests.

The main symbolic components are:

Syntactic soundness (r_{AST}) rewards patches that parse into well-formed syntax trees and penalizes malformed generations (Wang et al., 2021).

Structural fidelity ($r_{\text{CFG}}, r_{\text{DFG}}$) measures whether control- and data-flow structure remains consistent with the intended repair (Liang et al., 2019).

Security and execution evidence ($r_{\text{Semgrep}}, r_{\text{exec}}$) rewards patches that remove detected vulnerability patterns and, when public tests are available, execute successfully.

Together these signals guide the model toward patches that are syntactically valid, structurally plausible, security-aware, and executable. The reward is intentionally composite: no single signal is assumed to be sufficient across all benchmarks. For example, public tests are highly informative when present, while Semgrep and static validity checks remain available for security settings where executable tests may be absent.

2.3 Adaptive Training with a Router Model

The Router Model decides whether each batch should update the Repair Agent through SFT or RFT. For a mini-batch $B \subset \mathcal{D}$, we compute a compiler-derived routing feature vector $z_B = T([f_1(B), \dots, f_k(B)])$, where $T(\cdot)$ normalizes features such as code length, AST complexity, CFG depth, and current training behavior. A router $\rho_\omega(d \mid z_B)$ then outputs a binary decision $d_B \in \{0, 1\}$, where $d_B = 0$ selects SFT and $d_B = 1$ selects RFT. Appendix E gives the architecture and optimization details.

The two update paths optimize different signals. For SFT-routed batches, the model minimizes the standard negative log-likelihood $\mathcal{L}_{\text{SFT}}(B) = -\sum_{(x_i, y_i) \in B} \log \pi_\theta(y_i \mid x_i)$. For RFT-routed batches, the model samples candidates, scores them with $r(x, \hat{y})$, and applies the RLOO objective described in Appendix A.2. The routing design reflects a simple repair prior: many examples are best handled by learning common edit patterns directly, while harder examples benefit from symbolic feedback. Routing therefore preserves the efficiency and stability of SFT while using RFT selectively for examples where compiler-informed feedback is likely to help.

Algorithm 1 Training Pipeline of SYNTHFIX

Require: Dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, policy π_θ , router ρ_ω

- 1: **for** each batch $B \subset \mathcal{D}$ **do**
- 2: $z_B \leftarrow T([f_1(B), \dots, f_k(B)])$ {Normalize routing features}
- 3: $d_B \sim \rho_\omega(\cdot \mid z_B)$ {0: SFT, 1: RFT}
- 4: **if** $d_B = 0$ **then**
- 5: Compute supervised loss $\mathcal{L}_{\text{SFT}}(B)$
- 6: Update θ with $\nabla \mathcal{L}_{\text{SFT}}(B)$
- 7: **else**
- 8: Sample candidates $\mathcal{C}_K(x_i)$ for each $x_i \in B$
- 9: Score candidates with $r(x_i, \hat{y}) = \lambda^\top s(x_i, \hat{y})$
- 10: Compute reward-fine-tuning loss $\mathcal{L}_{\text{RFT}}(B)$
- 11: Update θ with $\nabla \mathcal{L}_{\text{RFT}}(B)$
- 12: **end if**
- 13: **end for**

2.4 Integrated Training Algorithm

Algorithm 1 summarizes the training loop. For each batch, SYNTHFIX computes routing features, samples a route, and applies one update. This keeps comparisons with SFT-only and RFT-only baselines controlled: all methods share the same model, optimizer budget, splits, and warmup schedule, differing only in routing and symbolic feedback.

2.5 Symbolic-Guided Inference-Time Selection

At inference time, SYNTHFIX samples K candidate patches and ranks them using the same family of symbolic evidence used during training. Let $\hat{y}^{(0)}$ be the greedy decode and $\mathcal{C}_K(x)$ be the full candidate set. We first construct a valid subset $\mathcal{V}_K(x) = \{\hat{y} \in \mathcal{C}_K(x) : \text{PASSES AVAILABLE VALIDITY CHECKS}(x, \hat{y})\}$, where the checks may include parseability, compilation, public tests, or static security scans depending on the benchmark. Surviving candidates are scored by a selector $q(x, \hat{y}) = \eta^\top s(x, \hat{y})$, and the best symbolic candidate is $\hat{y}^* = \arg \max_{\hat{y} \in \mathcal{V}_K(x)} q(x, \hat{y})$. SYNTHFIX then applies a greedy floor: it returns $\hat{y}^*$ only if it scores above the greedy decode, and otherwise returns $\hat{y}^{(0)}$. This selection step is leak-free because q uses public tests and static signals only; held-out tests are reserved for final evaluation.

3 Experimental Design

We evaluate SYNTHFIX on three execution- and security-grounded repair benchmarks across five modern code language models. This section summarizes the models, datasets, metrics, and research questions; full hyperparameters are in Appendix B.

3.1 Models and Baselines

We fine-tune five code LLMs spanning 1.3B–7B parameters: DeepSeek-Coder-1.3B (Guo et al., 2024), Llama-3.2-3B (Dubey et al., 2024), Qwen3-4B (Yang et al., 2025), CodeLLaMA-7B (Roziere et al., 2023), and StarCoder2-7B (Lozhkov et al., 2024). For each base model, we compare SYNTHFIX with SFT-only and RFT-only baselines. All methods are trained fresh from the same base model with LoRA and matched data, optimizer, and training budgets.

3.2 Benchmarks and Evaluation Metrics

We evaluate our framework on three execution- and security-grounded benchmarks: (1) **pyrepair**, a Python execution-repair benchmark we construct from MBPP (Austin et al., 2021) (execution-validated bug injection for train/validation/test) augmented with QuixBugs (Lin et al., 2017) as a real-bug test set; (2) **CodeFlaws** for C (~4k defects, each shipping a test suite) (Tan et al., 2017); and (3) **SVEN** (He and Vechev, 2023), a security-repair benchmark. We use a fixed random seed for reproducible train/validation/test splits across all experiments.

Repair quality is assessed primarily using *deployable* metrics that measure whether a patch works: **functional pass@1** (the patched program compiles and passes its held-out test suite) on pyrepair and CodeFlaws, and **security-cleared rate** (a static analyzer, Semgrep, reports the vulnerability removed) on SVEN. Selection of best-of- K candidates is leak-free: candidates are ranked using only public tests and static symbolic signals, scored on held-out tests, and constrained by a greedy floor so the selected patch can never regress below greedy decoding. We additionally report **CodeBLEU** (Ren et al., 2020) as a diagnostic, noting that it saturates on these benchmarks and does not measure functional correctness.

All experiments were conducted on two NVIDIA A6000 GPUs.

3.3 Research Questions

Our evaluation is organized around four research questions:

RQ1: End-to-End Repair Quality. How does SYNTHFIX compare to SFT-only and RFT-only baselines on deployable repair metrics?

RQ2: Training- and Test-Time Modules. How much do the adaptive router and symbolic best-of- K selector contribute beyond the base policy?

RQ3: Generalization Across Defect Types. Are gains consistent across the functional and security benchmark families we evaluate?

RQ4: Pipeline Ablation. What is the contribution of the router, reward-driven training, and symbolic test-time selector to the final repair rate?

4 Result Analysis

This section reports end-to-end repair quality, then analyzes the router, the symbolic selector, benchmark-level generalization, pipeline ablations, and qualitative cases.

4.1 Overall Performance (RQ1 & RQ2)

End-to-end quality. To answer **RQ1**, we evaluate SYNTHFIX on *deployable* metrics: functional pass@1 on pyrepair and CodeFlaws, and security-cleared rate on SVEN. Table 1 compares SYNTHFIX with SFT-only and budget-matched RFT-only baselines across five code LLMs. SYNTHFIX improves over SFT on every benchmark/model pair, with gains of +12.9% to +26.6% on pyrepair, +20.5% to +53.6% on CodeFlaws, and up to +14.3% in security clearance (ties occur at the ceiling). RFT alone stays close to SFT and sometimes regresses under greedy decoding, showing that training-time RL by itself is not sufficient for reliable deployable repair.

Module contributions. Having established the end-to-end gain, we next answer **RQ2** by separating the *training-time* and *test-time* modules.

Table 2 separates the two modules in controlled diagnostics. On CodeFlaws, the adaptive router improves the trained policy over a fixed no-router schedule (50 vs. 46 solved). The larger gain comes at test time: symbolic best-of- K raises the same policy from 50 to 66 solved on CodeFlaws, and

Table 1: Main deployable repair results across five base models. Functional pass@1 is reported for pyrepair and CodeFlaws, security-cleared rate for SVEN, and Δ is the relative gain of SYNTHFIX over SFT; best values are in **bold**.

Model	pyrepair (Python, $n=115$)				CodeFlaws (C, $n=389$)				SVEN (security, $n=16$)			
	SFT	RFT	SYNTHFIX	Δ	SFT	RFT	SYNTHFIX	Δ	SFT	RFT	SYNTHFIX	Δ
DeepSeek-1.3B	68.7	69.6	87.0	+26.6%	12.3	10.5	15.2	+22.9%	87.5	87.5	100	+14.3%
Llama-3.2-3B	73.9	73.9	85.2	+15.3%	14.4	13.6	22.1	+53.6%	100	87.5	100	–
Qwen3-4B-Base	80.9	80.9	91.3	+12.9%	18.8	15.9	22.6	+20.5%	87.5	87.5	93.8	+7.1%
CodeLLaMA-7B	72.2	75.7	81.7	+13.3%	12.9	12.6	18.8	+46.0%	93.8	93.8	93.8	–
StarCoder2-7B	79.1	76.5	93.0	+17.6%	15.7	15.7	22.6	+44.3%	87.5	81.2	100	+12.5%

Table 2: Controlled module diagnostics for DeepSeek-1.3B ($K=16$). The router comparison isolates adaptive training on CodeFlaws, while the selector comparison measures greedy, random, symbolic best-of- K , and oracle outcomes; values are solved@1 and Δ is relative to the same benchmark’s greedy floor.

Module	Benchmark	Variant	Solved@1	Δ
Router	CodeFlaws	Fixed schedule (no router)	46 (11.8%)	—
	CodeFlaws	SYNTHFIX router	50 (12.9%)	+4
Selector	CodeFlaws	Greedy floor	50 (12.9%)	—
	CodeFlaws	Random pick from K	41 (10.5%)	−9
	CodeFlaws	Symbolic best-of- K	66 (17.0%)	+16
	CodeFlaws	Oracle@ K (upper bound)	86 (22.1%)	+36
	pyrepair	Greedy floor	79 (68.7%)	—
	pyrepair	Symbolic best-of- K	100 (87.0%)	+21
	pyrepair	Oracle@ K (upper bound)	104 (90.4%)	+25

from 79 to 100 solved on pyrepair. Randomly choosing from the CodeFlaws candidate pool is worse than greedy, so the improvement is not merely from sampling more patches; it comes from ranking candidates with symbolic signals and a greedy floor.

Figure 2 visualizes this test-time module on the same trained policy: symbolic best-of- K consistently outperforms greedy decoding and approaches the oracle@ K ceiling as more held-out bugs are evaluated. Panel (a) shows the pattern on CodeFlaws (+16 solved at $n=389$); panel (b) shows the same trend on pyrepair.

Selection efficiency. Because Table 2 isolates the selector on a single model, Table 3 generalizes to five base models by adding the oracle@ K ceiling and reporting how much headroom the symbolic selector captures. Selection recovers 81–94% of the oracle gap on pyrepair (avg. 87.4%), 38–70% on harder CodeFlaws C defects (avg. 53.5%), and 83.3% of aggregate security headroom on SVEN where nonzero headroom remains. This confirms the test-time benefit is consistent and model-agnostic, not an artifact of one base model, while the residual oracle gap quantifies headroom stronger candidate generation could still unlock.

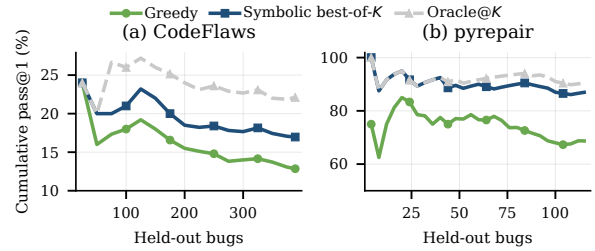


Figure 2: Cumulative test-time selection for DeepSeek-1.3B ($K=16$). Symbolic best-of- K stays above greedy on CodeFlaws and pyrepair, while oracle@ K shows the remaining reachable headroom.

The captured-headroom view is important because it separates two failure modes. If oracle@ K is low, the generator is not producing enough correct candidates; if oracle@ K is high but best-of- K is low, the selector is failing to identify them. SYNTHFIX mainly narrows the second gap, especially on pyrepair and SVEN, while CodeFlaws shows that harder C defects still need both better candidates and sharper ranking.

4.2 RQ3: Generalization Across Defect Types

We next examine whether the gains persist across functional Python repair, functional C repair, and security repair.

Table 4 gives a category-level view and adds the oracle@ K ceiling. SYNTHFIX improves the average score in every category. The captured-headroom column shows that pyrepair and SVEN are close to their sampled-candidate ceilings, while CodeFlaws leaves more unresolved oracle headroom. Because the current SVEN subset is dominated by CWE-89 (SQL injection), we report the benchmark-level security result rather than a fine-grained multi-CWE breakdown.

Auxiliary functionality diagnostic. Figure 3 reports a functionality-group analysis from the earlier FixJS/CodeFlaws exact-match setting. This figure is included only to give qualitative context about

Table 3: Test-time selection efficiency across all five base models ($K=16$). For each benchmark, we compare the greedy floor, symbolic best-of- K , and oracle@ K ceiling; Captured is the fraction of oracle headroom recovered by symbolic selection, and – indicates no oracle headroom.

Model	pyrepair (Python, $n=115$)				CodeFlaws (C, $n=389$)				SVEN (security, $n=16$)			
	Greedy	Best-of- K	Oracle@ K	Captured	Greedy	Best-of- K	Oracle@ K	Captured	Greedy	Best-of- K	Oracle@ K	Captured
DeepSeek-1.3B	68.7	87.0	90.4	84.0%	12.3	15.2	19.3	40.7%	87.5	100	100	100%
Llama-3.2-3B	73.9	85.2	87.8	81.3%	14.4	22.1	27.2	60.0%	100	100	100	–
Qwen3-4B-Base	80.9	91.3	93.0	85.7%	18.8	22.6	28.8	38.5%	87.5	93.8	100	50.0%
CodeLLaMA-7B	72.2	81.7	82.6	91.7%	12.9	18.8	21.3	69.7%	93.8	93.8	93.8	–
StarCoder2-7B	79.1	93.0	93.9	94.1%	15.7	22.6	27.5	58.7%	87.5	100	100	100%
Average	75.0	87.6	89.5	87.4%	14.8	20.3	24.8	53.5%	91.3	97.5	98.8	83.3%

Table 4: Category-level summary averaged over the five base models. The table groups the benchmark suite by repair property and reports SFT, RFT, SYNTHFIX, oracle@ K sampled-candidate ceilings, and the fraction of oracle headroom captured.

Benchmark category	n	SFT	RFT	SYNTHFIX	Oracle@ K	Captured
pyrepair (Python)	115	75.0	75.3	87.6	89.5	87.4%
CodeFlaws (C)	389	14.8	13.7	20.3	24.8	53.5%
SVEN (security)	16	91.3	87.5	97.5	98.8	83.3%
Avg. absolute gain over SFT	–	–1.5		+8.1	–	–

the kinds of programming patterns affected by repair models; it is not used for the main deployable-metric claims, which are supported by Table 1, Table 4, and Figure 2.

4.3 Pipeline Ablation (RQ4)

Finally, we ablate the main components of the SYNTHFIX pipeline. Because these controlled variants are run on CodeFlaws with DeepSeek-1.3B for tractability, Table 5 should be read as a focused component analysis rather than a full five-model sweep.

Table 5 makes three points. First, RFT-only training under greedy decoding does not improve over SFT (41 vs. 45 solved), so symbolic rewards alone are not enough. Second, replacing the router with a fixed schedule lowers the greedy policy (46 vs. 50 solved), while symbolic best-of- K still recovers

Table 5: Component ablation on CodeFlaws functional pass@1 (DeepSeek-1.3B, $n=389$, $K=16$). Rows remove or replace one major pipeline component while keeping the split and evaluation protocol fixed; best non-oracle result is in **bold**.

Variant	Selection rule	Solved@1
SFT-only baseline	Greedy	45 (11.6%)
RFT-only baseline	Greedy	41 (10.5%)
No router (fixed schedule)	Greedy	46 (11.8%)
No router (fixed schedule)	Symbolic best-of- K	64 (16.5%)
SYNTHFIX without test-time selector	Greedy	50 (12.9%)
SYNTHFIX with unranked candidates	Random pick from K	41 (10.5%)
SYNTHFIX full pipeline	Symbolic best-of- K	66 (17.0%)
Oracle@ K upper bound	Held-out oracle	86 (22.1%)

many repairs (64 solved). Third, sampling without symbolic ranking is insufficient: random pick falls to 41 solved, whereas the full selector solves 66. The oracle@ K value of 86 shows that stronger generation and ranking still leave headroom.

This ablation also clarifies the role of the greedy floor. The selector is not allowed to trade a known greedy repair for an unsupported sampled candidate unless the symbolic evidence justifies it. That design is conservative, but it is appropriate for repair: deployability depends more on avoiding regressions than on maximizing textual diversity.

4.4 Qualitative Case Study

We finish with representative repair cases. Figure 4 compares patches from SFT, RFT, and SYNTHFIX for two scenarios and illustrates why symbolic feedback is useful even when surface-form metrics are ambiguous.

Case Study 1: Cross-Site Scripting (CWE-79).

As shown in the left panel of Figure 4, a JavaScript web server is vulnerable to script injection. The **SFT** model applies a naive fix that only strips `<script>` tags, while the **RFT** model applies `escapeHtml(...)` but performs no other checks. By combining the pattern learning of SFT with

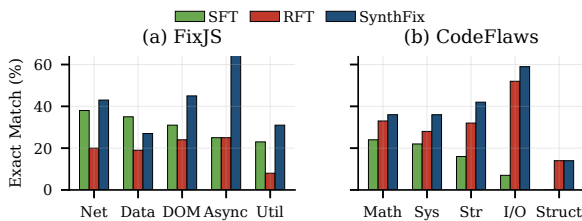


Figure 3: Auxiliary exact-match analysis by functionality group in the FixJS/CodeFlaws diagnostic setting. This figure is retained for qualitative context; main claims use deployable metrics.

Vulnerable server.js (CWE-79)

```
const http = require('http');
const url = require('url');
const escapeHtml = require('escape-html');
http.createServer((req, res) => {
  const query = url.parse(req.url, true).query;
  const userInput = query.user || '';
  const responseHtml = `<html><body><div>Your input: ${userInput}</div></body></html>`;
  res.writeHead(200, { 'Content-Type': 'text/html' });
  res.end(responseHtml);
}).listen(3000);
```

Dynamic Repaired server.js

```
- const userInput = query.user || '';
+ let userInput = (query.user || "").replace(/<script>/g, "").replace(/</script>/g, "");
```

Static Repaired server.js

```
- const userInput = query.user || '';
+ let userInput = escapeHtml(query.user || "");
```

SynthFix Repaired server.js

```
- const userInput = query.user || '';
+ let userInput = filter(escapeHtml(limit(query.user || "")));
```

Buggy sumPositive.js (Functionality Error)

```
function sumPositive(arr) {
  let total = 0;
  for (let i = 0; i < arr.length; i++) {
    total += arr[i];
  }
  return total;
}
console.log(sumPositive([1, -2, 3, 'hello', 0]));
```

Dynamic Repaired sumPositive.js

```
- total += arr[i];
+ if(typeof arr[i] === 'number') total += arr[i];
```

Static Repaired sumPositive.js

```
- total += arr[i];
+ if(typeof arr[i] === 'number' && arr[i] > 0) total += arr[i];
```

SynthFix Repaired sumPositive.js

```
- total += arr[i];
+ if(typeof arr[i] === 'number' && arr[i] > 0 && Number.isFinite(arr[i])) total += arr[i];
```

Figure 4: Representative repairs from SFT, RFT, and SYNTHFIX. The examples show a security sanitization case and a logic repair where symbolic evidence helps select the deployable patch.

symbolic validation, SYNTHFIX generates a multi-layered defense:

```
let userInput = filter(escapeHtml(limit(query.user || '')));
```

This repair composes length limiting, HTML escaping, and allow-list filtering, which is more robust than either standalone baseline.

Case Study 2: Array Summation Logic Error.

The right panel of Figure 4 shows a function that incorrectly sums array elements. The SFT model adds a partial type check but still allows negative numbers. The RFT model produces a correct but verbose condition with a redundant check. SYNTHFIX synthesizes the concise repair:

```
if (typeof arr[i] === 'number' && arr[i] > 0)
  total += arr[i];
```

This succinct condition enforces the intended semantics without unnecessary checks, illustrating how the adaptive pipeline can produce repairs that are both compact and correct.

5 Discussion

Our results show that supervised learning, symbolic reinforcement feedback, and symbolic test-time selection play complementary roles in automated program repair. SFT provides a stable foundation for common repair patterns, while RFT targets examples that need stronger semantic evidence. This mirrors a practical repair workflow: known fixes can often be applied directly, but harder bugs re-

quire additional feedback from tests, analyzers, or compiler behavior.

Adaptive training. The module analysis suggests that repair benefits from allocating training effort based on code-level difficulty. Some bugs are formulaic and can be learned well through imitation, while others require feedback-driven exploration. In the controlled CodeFlaws study, the router improves the trained policy over a fixed schedule, showing that a single uniform objective is not always the best use of the training budget.

Inference-time validation. SYNTHFIX also highlights the importance of symbolic feedback after training. The largest deployable gains come from combining adaptive symbolic training with symbolic best-of- K selection (Table 1, Figure 2). The component ablation in Table 5 shows that neither reward-driven training nor candidate sampling is sufficient alone; the strongest result comes from coupling generation with evidence-based selection.

Deployment implication. This matters because stronger code LLMs may produce more plausible candidate patches, but plausibility is not a deployment criterion. A repair workflow still needs evidence that a patch compiles, preserves intended behavior, and removes the vulnerability. SYNTHFIX treats this evidence as part of the decision process rather than only an after-the-fact evaluation metric, making the repair pipeline more aligned with deployable software maintenance.

6 Related Work

Our work combines automated program repair, neural code generation, reinforcement learning, and compiler-informed analysis. We organize prior work around the same design choices as SYNTHFIX: SFT-based repair, feedback-driven refinement, symbolic analysis, and inference-time validation.

6.1 Neural Program Repair with SFT

Neural and LLM-based methods have become central to automated code repair (Allamanis et al., 2018; Chen et al., 2021; Li et al., 2024; Bansal et al., 2023; Zhang, 2022; Zhang and Leach, 2025; Zhang et al., 2025a; Richter et al., 2022; Feng et al., 2020). Most use Supervised Fine-Tuning (SFT) to learn mappings from buggy to fixed code from large repair datasets (Yin and Neubig, 2017; Hayati et al., 2018; Habib and Pradel, 2019; Li et al., 2019; Gupta et al., 2019; Allamanis et al., 2021; Jiang et al., 2023). While effective for common syntactic patterns, these methods can still generate plausible patches that are functionally incorrect or fail to remove the vulnerability (Berabi et al., 2021; Huang et al., 2023; Pailoor et al., 2024; Zhang et al., 2022a; Kou et al., 2024; Ziems and Wu, 2021).

6.2 Reinforcement Learning for Code

Reward Fine-Tuning (RFT) and reinforcement learning address this limitation by optimizing models with dynamic feedback, such as unit tests or security checks (Le et al., 2022; Shojaee et al., 2023; Fang et al., 2025). PPO is commonly used for iterative refinement (Schulman et al., 2017), and recent work incorporates multi-step symbolic verification signals (Zhang et al., 2025b). However, these methods can be computationally expensive and slow to converge, limiting practicality for large-scale repair (Huang et al., 2023; Shi et al., 2023).

6.3 Symbolic and Compiler-Informed Analysis

Compiler-informed repair methods use representations such as ASTs and CFGs to capture structural and semantic properties of code (Shi et al., 2020; Wu et al., 2022; Jiang et al., 2018; Klieber et al., 2021; Mandal et al., 2018; Wang et al., 2021). These signals are precise and useful, but many traditional systems depend on predefined heuristics or manually crafted templates, which can limit flexibility on diverse real-world vulnerabilities (Zhang et al., 2022b).

6.4 Inference-Time Validation and Selection

Recent repair pipelines increasingly separate candidate generation from candidate acceptance when execution or validation feedback is available (Gupta et al., 2020; Le et al., 2022; Shojaee et al., 2023; Kulsum et al., 2024). This distinction is important because the highest-probability patch may not compile, pass tests, or remove the vulnerability. Generate-and-rank workflows therefore use symbolic evidence to filter invalid patches, score candidates with public tests or static analyzers, and compare alternatives against a greedy baseline. Prior work on developer validation of LLM-generated code similarly shows that explicit checking is needed beyond surface plausibility (Tang et al., 2024). Larger candidate pools can increase oracle headroom, but can also introduce regressions if the final choice is random or based only on likelihood, motivating the greedy floor and symbolic reranker in SYNTHFIX.

Neuro-symbolic positioning. SYNTHFIX differs from methods that use symbolic checks only as a final filter. Symbolic evidence appears in three places: it shapes reward for feedback-driven updates, supplies routing features for deciding when such updates are useful, and ranks candidate patches at inference time. The goal is not to replace neural repair with a hand-written analyzer, but to keep neural generation anchored to evidence that corresponds to deployable repair behavior.

7 Conclusion and Future Work

We introduced SYNTHFIX, an adaptive neuro-symbolic framework for automated vulnerability repair. By routing training between SFT and RFT and reusing symbolic evidence for best-of- K selection, SYNTHFIX aligns neural patch generation with compiler-, analyzer-, and test-facing repair signals. Across five code LLMs and three benchmarks, it improves over SFT-only and RFT-only baselines; module analyses show that the gains come from the full training-and-selection pipeline rather than sampling alone. These results suggest that effective repair systems should not rely only on larger generators, but should also use symbolic evidence to decide when models should learn from feedback, how candidate patches should be ranked, and which repair is safe enough to deploy. More broadly, SYNTHFIX shows that neural generation and symbolic validation can be integrated throughout the repair pipeline rather than treated as separate stages.

Limitations

A primary limitation is the Router Model’s reliance on static code features (e.g., AST complexity, CFG depth). These features are inexpensive and useful, but they miss dynamic runtime behavior that could better guide complex repairs involving subtle execution logic. In addition, our evaluation uses established academic benchmarks. While standard for this area, they may not reflect the scale and dependencies of large industrial systems. Thus, SYNTHFIX’s performance in such settings remains future work.

Threats to validity. *Construct:* surface-overlap metrics (CodeBLEU/exact match) saturate and do not track whether a patch runs (Table 8), so our claims rely on execution- and security-grounded metrics (functional pass@1, Semgrep-verified clearance). *Internal:* we train SFT, RFT, and SYNTHFIX fresh from the same base model with identical splits, optimizer, and budget, and keep best-of- K selection leak-free through a public-test/static-signal-only reranker with a greedy floor, so gains cannot come from held-out test access; the module analysis separates the router and symbolic selector from larger candidate pools. *External:* our deepest ablations use DeepSeek-1.3B on CodeFlaws for tractability, but the headline comparison spans five model families (1.3B–7B) and three benchmarks, with SYNTHFIX either improving over SFT or tying at a ceiling in every benchmark/model cell (Table 1); generalization to multi-file industrial repositories remains future work.

Ethical Considerations

While SYNTHFIX aims to improve software security, we note the risks common to any automated repair tool. Chief among them is a false sense of security: a generated patch may be incomplete or introduce subtle runtime errors, so over-reliance could reduce manual auditing. We therefore intend SYNTHFIX as an assistive tool that augments—rather than replaces—developer oversight, and advocate deploying it only within workflows that mandate testing and human review of all generated patches.

Use of AI Assistants. The authors used AI assistants (LLMs) for minor editorial assistance, including language polishing and LaTeX formatting. All research ideation, experimental design, implementation, analysis, and claims are the authors’ own.

References

- Miltiadis Allamanis, Earl T Barr, Premkumar Devanbu, and Charles Sutton. 2018. A survey of machine learning for big code and naturalness. *ACM Computing Surveys (CSUR)*, 51(4):1–37.
- Miltiadis Allamanis, Henry Jackson-Flux, and Marc Brockschmidt. 2021. Self-supervised bug detection and repair. *Advances in Neural Information Processing Systems*, 34:27865–27876.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and 1 others. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.
- Aakash Bansal, Chia-Yi Su, Zachary Karas, Yifan Zhang, Yu Huang, Toby Jia-Jun Li, and Collin McMillan. 2023. Modeling programmer attention as scanpath prediction. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1732–1736. IEEE.
- Berkay Berabi, Jingxuan He, Veselin Raychev, and Martin Vechev. 2021. Tfix: Learning to fix coding errors with a text-to-text transformer. In *International Conference on Machine Learning*, pages 780–791. PMLR.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, and 1 others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- David de Fitero-Dominguez, Eva Garcia-Lopez, Antonio Garcia-Cabot, and Jose-Javier Martinez-Herrera. 2024. Enhanced automated code vulnerability repair using large language models. *Engineering Applications of Artificial Intelligence*, 138:109291.
- Guanting Dong, Hongyi Yuan, Keming Lu, Chengpeng Li, Mingfeng Xue, Dayiheng Liu, Wei Wang, Zheng Yuan, Chang Zhou, and Jingren Zhou. 2023. How abilities in large language models are affected by supervised fine-tuning data composition. *arXiv preprint arXiv:2310.05492*.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, and 1 others. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Zihan Fang, Yifan Zhang, Yueke Zhang, Kevin Leach, and Yu Huang. 2025. Dpo-f+: Aligning code repair feedback with developers’ preferences. *arXiv preprint arXiv:2511.01043*.
- Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and 1 others. 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155*.

- Michael Fu, Chakkrit Tantithamthavorn, Trung Le, Van Nguyen, and Dinh Phung. 2022. Vulrepair: a t5-based automated software vulnerability repair. In *Proceedings of the 30th ACM joint european software engineering conference and symposium on the foundations of software engineering*, pages 935–947.
- Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y Wu, YK Li, and 1 others. 2024. Deepseek-coder: When the large language model meets programming – the rise of code intelligence. *arXiv preprint arXiv:2401.14196*.
- Kavi Gupta, Peter Ebert Christensen, Xinyun Chen, and Dawn Song. 2020. Synthesize, execute and debug: Learning to repair for neural program synthesis. *Advances in Neural Information Processing Systems*, 33:17685–17695.
- Rahul Gupta, Aditya Kanade, and Shirish Shevade. 2019. Neural attribution for semantic bug-localization in student programs. *Advances in Neural Information Processing Systems*, 32.
- Andrew Habib and Michael Pradel. 2019. Neural bug finding: A study of opportunities and challenges. *arXiv preprint arXiv:1906.00307*.
- Shirley Anugrah Hayati, Raphael Olivier, Pravalika Avvaru, Pengcheng Yin, Anthony Tomasic, and Graham Neubig. 2018. Retrieval-based neural code generation. *arXiv preprint arXiv:1808.10025*.
- Jingxuan He and Martin Vechev. 2023. Large language models for code: Security hardening and adversarial testing. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 1865–1879.
- Kai Huang, Xiangxin Meng, Jian Zhang, Yang Liu, Wenjie Wang, Shuhao Li, and Yuqing Zhang. 2023. An empirical study on fine-tuning large language models of code for automated program repair. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1162–1174. IEEE.
- Nafis Tanveer Islam and Peyman Najafirad. 2024. Code security vulnerability repair using reinforcement learning with large language models. *arXiv preprint arXiv:2401.07031*.
- Jiajun Jiang, Yingfei Xiong, Hongyu Zhang, Qing Gao, and Xiangqun Chen. 2018. Shaping program repair space with existing patches and similar code. In *Proceedings of the 27th ACM SIGSOFT international symposium on software testing and analysis*, pages 298–309.
- Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of code language models on automated program repair. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 1430–1442. IEEE.
- William Klieber, Ruben Martins, Ryan Steele, Matt Churilla, Mike McCall, and David Svoboda. 2021. Automated code repair to ensure spatial memory safety. In *2021 IEEE/ACM International Workshop on Automated Program Repair (APR)*, pages 23–30. IEEE.
- Bonan Kou, Shengmai Chen, Zhijie Wang, Lei Ma, and Tianyi Zhang. 2024. Do large language models pay similar attention like human programmers when generating code? *Proceedings of the ACM on Software Engineering*, 1(FSE):2261–2284.
- Ummay Kulsum, Haotian Zhu, Bowen Xu, and Marcelo d’Amorim. 2024. A case study of llm for automated vulnerability repair: Assessing impact of reasoning and patch validation feedback. In *Proceedings of the 1st ACM International Conference on AI-Powered Software*, pages 103–111.
- Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven Chu Hong Hoi. 2022. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. *Advances in Neural Information Processing Systems*, 35:21314–21328.
- Jiliang Li, Yifan Zhang, Zachary Karas, Collin McMillan, Kevin Leach, and Yu Huang. 2024. Do machines and humans focus on similar code? exploring explainability of large language models in code summarization. In *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension*, pages 47–51.
- Yi Li, Shaohua Wang, Tien N Nguyen, and Son Van Nguyen. 2019. Improving bug detection via context-based code representation learning and attention-based neural networks. *Proceedings of the ACM on Programming Languages*, 3(OOPSLA):1–30.
- Hongliang Liang, Yuxing Yang, Lu Sun, and Lin Jiang. 2019. Jsac: A novel framework to detect malicious javascript via cnns over ast and cfg. In *2019 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE.
- Derrick Lin, James Koppel, Angela Chen, and Armando Solar-Lezama. 2017. Quixbugs: A multi-lingual program repair benchmark set based on the quixey challenge. In *Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity*, pages 55–56.
- Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, and 1 others. 2024. Starcoder 2 and the stack v2: The next generation. *arXiv preprint arXiv:2402.19173*.
- Avijit Mandal, Devina Mohan, Raoul Jetley, Sreeja Nair, and Meenakshi D’Souza. 2018. A generic static analysis framework for domain-specific languages. In *2018 IEEE 23rd International Conference on Emerging Technologies and Factory Automation (ETFA)*, volume 1, pages 27–34. IEEE.

- Shankara Pailoor, Yuepeng Wang, and Işıl Dillig. 2024. Semantic code refactoring for abstract data types. *Proceedings of the ACM on Programming Languages*, 8(POPL):816–847.
- Emilio Parisotto, Abdel-rahman Mohamed, Rishabh Singh, Lihong Li, Dengyong Zhou, and Pushmeet Kohli. 2016. Neuro-symbolic program synthesis. *arXiv preprint arXiv:1611.01855*.
- Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. Codebleu: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297*.
- Cedric Richter, Jan Haltermann, Marie-Christine Jakobs, Felix Pauck, Stefan Schott, and Heike Wehrheim. 2022. Are neural bug detectors comparable to software developers on variable misuse bugs? In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, pages 1–12.
- Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, and 1 others. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- Zhidong Shen and Si Chen. 2020. A survey of automatic software vulnerability detection, program repair, and defect prediction techniques. *Security and Communication Networks*, 2020(1):8858010.
- Ensheng Shi, Yanlin Wang, Hongyu Zhang, Lun Du, Shi Han, Dongmei Zhang, and Hongbin Sun. 2023. Towards efficient fine-tuning of pre-trained code models: An experimental study and beyond. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 39–51.
- Ke Shi, Yang Lu, Jingfei Chang, and Zhen Wei. 2020. Pathpair2vec: An ast path pair-based code representation method for defect prediction. *Journal of Computer Languages*, 59:100979.
- Parshin Shojaei, Aneesh Jain, Sindhu Tipirneni, and Chandan K Reddy. 2023. Execution-based code generation using deep reinforcement learning. *arXiv preprint arXiv:2301.13816*.
- Shin Hwei Tan, Jooyong Yi, Sergey Mechtaev, Abhik Roychoudhury, and 1 others. 2017. Codeflaws: a programming competition benchmark for evaluating automated program repair tools. In *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*, pages 180–182. IEEE.
- Ningzhi Tang, Meng Chen, Zheng Ning, Aakash Bansal, Yu Huang, Collin McMillan, and Toby Jia-Jun Li. 2024. Developer behaviors in validating and repairing llm-generated code using ide and eye tracking. In *2024 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, pages 40–46. IEEE.
- Xin Wang, Yasheng Wang, Fei Mi, Pingyi Zhou, Yao Wan, Xiao Liu, Li Li, Hao Wu, Jin Liu, and Xin Jiang. 2021. Syncobert: Syntax-guided multi-modal contrastive pre-training for code representation. *arXiv preprint arXiv:2108.04556*.
- Bingting Wu, Bin Liang, and Xiaofang Zhang. 2022. Turn tree into graph: Automatic code review via simplified ast driven graph convolutional network. *Knowledge-Based Systems*, 252:109450.
- An Yang, Anfeng Li, Baosong Yang, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- Pengcheng Yin and Graham Neubig. 2017. A syntactic neural model for general-purpose code generation. *arXiv preprint arXiv:1704.01696*.
- Yifan Zhang. 2022. Leveraging artificial intelligence on binary code comprehension. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, pages 1–3.
- Yifan Zhang, Chen Huang, Kevin Cao, Yueke Zhang, Scott Thomas Andersen, Huajie Shao, Kevin Leach, and Yu Huang. 2022a. Pre-training representations of binary code using contrastive learning. *arXiv preprint arXiv:2210.05102*.
- Yifan Zhang, Chen Huang, Zachary Karas, Thuy Dung Nguyen, Kevin Leach, and Yu Huang. 2025a. Enhancing code llm training with programmer attention. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, pages 616–620.
- Yifan Zhang and Kevin Leach. 2025. Leveraging human insights for enhanced llm-based code repair. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, pages 1536–1537.
- Yifan Zhang, Junwen Yang, Haoyu Dong, Qingchen Wang, Huajie Shao, Kevin Leach, and Yu Huang. 2022b. Astro: An ast-assisted approach for generalizable neural clone detection. *arXiv preprint arXiv:2208.08067*.
- Yueke Zhang, Yifan Zhang, Kevin Leach, and Yu Huang. 2025b. Codegrad: Integrating multi-step verification with gradient-based llm refinement. *arXiv preprint arXiv:2508.10059*.
- Noah Ziems and Shaoen Wu. 2021. Security vulnerability detection using deep learning natural language processing. In *IEEE INFOCOM 2021-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, pages 1–6. IEEE.

A Technical Background for SYNTHFIX

This appendix provides implementation details that support the main paper. We briefly define the SFT/RFT objectives used by SYNTHFIX, then document the model configuration, reward signals, inference-time selector, benchmark construction, and auxiliary diagnostics.

A.1 Supervised Fine-Tuning (SFT)

Supervised Fine-Tuning is a standard technique for adapting pre-trained models to specific tasks. Given a dataset of buggy code snippets x_i and their corresponding ground-truth fixes y_i , SFT optimizes the model’s parameters θ by minimizing the negative log-likelihood of the target sequences: $\mathcal{L}_{\text{SFT}}(\theta) = -\sum_i \log P_\theta(y_i | x_i)$.

In SYNTHFIX, SFT provides the stable imitation pathway for learning common repair patterns. It also defines the warmup checkpoint used by all methods before the reward-driven stage, ensuring that SFT-only, RFT-only, and SYNTHFIX share the same supervised starting point.

A.2 Reward Fine-Tuning (RFT)

Reward Fine-Tuning uses reinforcement learning to optimize a model based on feedback from a reward function, which is particularly useful when a ground-truth output is not available or when optimizing for complex criteria beyond surface-level similarity. We instantiate RFT with a variance-reduced REINFORCE objective using a *leave-one-out* (RLOO) baseline. For each prompt x , the policy samples a group of k candidate patches $\{\hat{y}^{(1)}, \dots, \hat{y}^{(k)}\}$; each candidate is scored by the symbolic reward $r(\hat{y}^{(j)})$, and its advantage is computed against the mean reward of the *other* candidates in the group, $\hat{A}^{(j)} = r(\hat{y}^{(j)}) - \frac{1}{k-1} \sum_{l \neq j} r(\hat{y}^{(l)})$. To keep the policy from drifting away from the fluent SFT reference and collapsing, we add a KL-divergence penalty to the frozen SFT reference policy π_{ref} , giving the compact objective $\mathcal{L}_{\text{RFT}}(\theta) = -\mathbb{E}[\hat{A}^{(j)} \log \pi_\theta(\hat{y}^{(j)} | x)] + \beta_{\text{KL}} \text{KL}(\pi_\theta \| \pi_{\text{ref}})$. This avoids training a separate value network while retaining low-variance updates. In SYNTHFIX, RFT is the feedback-driven pathway: it exposes candidate patches to symbolic feedback about syntax, compilation, public tests, security scans, and repair-oriented edits. Exact RL hyperparameters are given in Appendix B.

A.3 Symbolic Representations for Reward and Routing

Our framework integrates symbolic reasoning by leveraging compiler-informed code representations, primarily Abstract Syntax Trees (ASTs), Control Flow Graphs (CFGs), and data-flow features.

An **Abstract Syntax Tree (AST)** represents hierarchical syntactic structure. A **Control Flow Graph (CFG)** models possible execution paths, and data-flow features summarize how values are produced and consumed.

These representations serve a dual purpose in SYNTHFIX:

1. **For adaptive routing:** Compiler-derived metrics such as code length, AST complexity, and CFG depth form the feature vector f_B used by the Router Model to choose between SFT and RFT updates.
2. **For reward and selection:** The same families of evidence are used to score candidate patches during RFT and to rank sampled candidates at inference time. We assess a candidate patch $\hat{y}$ with a composite reward over static structure, lint/compile validity, security scanning, repair-effect signals, and, when available, public execution tests: $r(\hat{y}) = \sum_c \lambda_c r_c(\hat{y})$. Each component provides distinct feedback, and the exact instantiation used in the experiments is described in Appendix B.3.

B Detailed Experimental Setup

This appendix provides the model configurations, two-stage training recipe, symbolic reward implementation, and inference-time selection procedure used to evaluate SYNTHFIX, supplementing Section 3. All settings are identical across the five base models unless noted in Table 6; SFT-only, RFT-only, and SYNTHFIX use the same splits, LoRA setup, optimizer budget, and decoding budget so that comparisons are controlled.

B.1 Repair Agent and Parameter-Efficient Fine-Tuning

Each Repair Agent is the corresponding pre-trained base model (DeepSeek-Coder-1.3B, Llama-3.2-3B, Qwen3-4B-Base, CodeLLaMA-7B, StarCoder2-7B), adapted with **Low-Rank Adaptation (LoRA)** rather than full fine-tuning. We use a LoRA rank

Table 6: Per-model configuration and training budget. All models use the same LoRA rank, optimizer, four-epoch schedule, and $K=16$ decoding; gradient checkpointing (GC) is enabled for larger models.

Base model	Params	Batch size	GC
DeepSeek-Coder-1.3B	1.3B	16	—
Llama-3.2-3B	3B	8	—
Qwen3-4B-Base	4B	8	✓
CodeLLaMA-7B	7B	4	✓
StarCoder2-7B	7B	4	✓

of $r=16$ with scaling $\alpha=32$ and dropout 0.05, applied to the attention projections (q_proj, k_proj, v_proj, o_proj); base weights stay frozen. This keeps the trainable footprint below 1% of model parameters and lets all five models—including the 7B variants—fit on two NVIDIA A6000 (48 GB) GPUs. Inputs are tokenized with the model’s native tokenizer using left-side padding and a maximum sequence length of 512 tokens. Per-model batch sizes and decode lengths are listed in Table 6.

B.2 Two-Stage Training Recipe

SYNTHFIX is trained *fresh* from each base model in two stages, for a total of four epochs:

Stage 1 — SFT warmup (2 epochs). We first minimize the standard supervised loss $\mathcal{L}_{\text{SFT}}$ (Appendix A.1) on the (buggy, fixed) pairs to establish a fluent repair prior. The same warmed-up checkpoint defines the SFT-only baseline and the frozen reference policy π_{ref} used by the KL anchor in Stage 2.

Stage 2 — router-gated symbolic RL (2 epochs).

We then continue training with the router-gated RLOO objective of Appendix A.2. For each prompt the policy samples a group of $k=2$ candidates; advantages use the leave-one-out baseline and the update is regularized toward π_{ref} .

The Repair Agent’s adapters are optimized with **AdamW** (learning rate 2×10^{-4} , weight decay 0.01, gradient-norm clipping at 1.0). During Stage 2, RL rollouts are decoded with temperature 0.95, top- p 0.95, and a no-repeat 3-gram constraint to encourage candidate diversity; the RL term and the KL penalty use weights $\beta_{\text{RL}}=0.12$ and $\beta_{\text{KL}}=0.12$. The Router Model (Appendix E) is optimized concurrently with a separate Adam optimizer at learning rate 1×10^{-3} . The RFT-only baseline uses the same Stage-2 objective but routes every batch

through the RL pathway (no adaptive router). Final evaluation is always performed with the held-out test split and the leak-free selector described below.

B.3 Symbolic Reward Implementation

The composite reward $r(\hat{y}) = \sum_c \lambda_c r_c(\hat{y})$ aggregates complementary symbolic signals, each normalized to $[0, 1]$:

r_{AST} (**syntactic soundness**). A parse of the candidate; full credit for a well-formed AST and a steep penalty for non-parseable code.

$r_{\text{CFG}} / r_{\text{DFG}}$ (**structural/data-flow fidelity**).

Graph-similarity scores between the candidate and reference control-flow and data-flow graphs, combining node, edge, and path overlap.

r_{lint} (**static validity**). Linter/compiler-frontend signal rewarding code that passes static checks without warnings.

r_{Semgrep} (**security**). A Semgrep scan with a custom ruleset; the score starts at 100 and is decremented for each matched vulnerability pattern.

r_{exec} (**execution**). When an executable suite is available, the fraction of *public/visible* tests the patched program passes (held-out tests are never used in the reward).

$r_{\text{repair-effect}}$. A reference-free term that rewards repair-directed edits (added guards, bounds checks, sanitizers, error handling) relative to the buggy input.

On execution benchmarks, the reward gives the largest weight to public-test execution and repair-effect signals, while retaining parse, data-flow, lint, CFG, AST, and surface-similarity terms. On static/security settings, execution is unavailable, so the reward relies on parse/lint/structure, Semgrep, and similarity signals. Reward computation is fully procedural (no learned reward model), which makes it deterministic and inexpensive to reproduce; the focused component ablation in the main text tests the pipeline modules rather than trying to rank individual reward weights.

B.4 Inference-Time Best-of- K Selection

At decoding time SYNTHFIX samples $K=16$ candidates per bug using one greedy decode plus sampled decodes spanning temperatures 0.2–1.0 with

Algorithm 2 Symbolic-Guided Inference-Time Selection

Require: bug x , policy π_θ , reranker g , budget K

- 1: $\hat{y}_0 \leftarrow \text{GREEDYDECODE}(\pi_\theta, x)$
- 2: $\mathcal{C} \leftarrow \{\hat{y}_0\} \cup \text{SAMPLEK}(\pi_\theta, x, K-1)$ {mixed temps}
- 3: $\mathcal{V} \leftarrow \{\hat{y} \in \mathcal{C} : \text{PASSESAVAILABLEVALIDITYCHECKS}(\hat{y})\}$
- 4: **if** $\mathcal{V} = \emptyset$ **then**
- 5: **return** $\hat{y}_0$ {greedy floor}
- 6: **end if**
- 7: $\hat{y}^* \leftarrow \arg \max_{\hat{y} \in \mathcal{V}} g(\phi(\hat{y}))$ {rank by symbolic features}
- 8: **return** $g(\phi(\hat{y}^*)) \geq g(\phi(\hat{y}_0)) ? \hat{y}^* : \hat{y}_0$

top- p 0.95. Candidates that fail available hard validity checks—such as parsing, compilation, or public/visible tests—are discarded before ranking. Surviving candidates are scored by a lightweight reranker over symbolic features: parseability, lint/compile status, CFG/DFG similarity, Semgrep score, public-test pass rate when available, candidate temperature, and average token log-probability. The top-ranked candidate is returned subject to a **greedy floor**: if no sampled candidate outranks the greedy decode, the greedy decode is kept. Crucially, ranking consults only public tests and static signals; held-out tests are reserved solely for final scoring. Algorithm 2 summarizes the procedure.

C Benchmark Construction and Statistics

This appendix details how each benchmark is built and split. All splits use a fixed random seed for reproducibility, and we deduplicate across splits to prevent train/test leakage.

pyrepair (Python, execution). We construct pyrepair from two sources. For training/validation/test we apply *execution-validated bug injection* to MBPP (Austin et al., 2021) problems: we introduce small, behavior-changing mutations (operator swaps, off-by-one edits, boundary and conditional perturbations) and keep only those whose buggy version fails at least one of the problem’s tests while the original passes, yielding (buggy, fixed) pairs each shipping an executable test suite. To measure transfer to *naturally occurring* defects, the held-out test set additionally includes QuixBugs (Lin et al., 2017) programs ($n=40$ classic algorithmic

Table 7: Scaling of inference-time selection on CodeFlaws (DeepSeek-1.3B, $K=16$). Values are functional pass@1 (%) over the first n evaluated bugs, comparing greedy, random, symbolic best-of- K , and oracle selection.

n	SFT greedy	SF greedy	Random pick	Best-of- K (ours)	Oracle
75	14.7	17.3	14.7	20.0	26.7
150	14.0	18.0	14.0	22.0	26.0
225	12.9	15.1	12.0	18.2	23.1
300	12.3	14.0	10.7	17.7	22.7
375	11.7	13.1	10.4	17.1	21.9

bugs); functional pass@1 is reported on the combined held-out set ($n=115$).

CodeFlaws (C, execution). We use CodeFlaws (Tan et al., 2017), $\sim 4k$ real defects mined from Codeforces submissions, each accompanied by a test suite. After filtering items that fail to build in our sandbox, the held-out test set contains $n=389$ defects. Functional pass@1 requires the patched program to compile and pass its held-out tests.

SVEN (security). We use the SVEN (He and Vechev, 2023) security-repair corpus of real vulnerable/fixed C and Python functions spanning common CWEs. The security-cleared rate is the fraction of held-out items ($n=16$) for which Semgrep reports the original vulnerability removed in the generated patch.

For all benchmarks, the symbolic reward and the inference reranker see only the training/public signals; held-out tests and the security scan on the held-out set are reserved for evaluation.

D Additional Results

Scaling of inference-time selection. Table 7 reports how the four decoding strategies behave as the number of evaluated CodeFlaws bugs grows (DeepSeek-1.3B, $K=16$). The ordering is stable from early on: symbolic best-of- K consistently exceeds greedy, random pick consistently trails it, and the oracle marks the achievable headroom. This supports the main-text selector analysis by showing that the result is not an artifact of a particular subset size.

Surface-similarity metrics saturate. Table 8 reports greedy-decoding CodeBLEU for the SFT-only, RFT-only, and SYNTHFIX models (all trained

Table 8: Greedy-decoding CodeBLEU (%) for SFT, RFT, and SYNTHFIX. Δ is SYNTHFIX–SFT, showing that overlap metrics change only weakly even when deployable repair outcomes improve; best values are in **bold**.

Model	Bench.	SFT	RFT	SYNTHFIX	Δ
DeepSeek	pyrepair	77.58	77.78	77.53	−0.05
DeepSeek	SVEN	43.88	43.81	43.73	−0.15
DeepSeek	CodeFlaws	57.90	57.25	57.87	−0.03
Llama-3.2	pyrepair	76.28	76.19	77.30	+1.02
Llama-3.2	SVEN	53.98	53.52	54.52	+0.54
Llama-3.2	CodeFlaws	70.38	70.43	70.45	+0.07
Qwen3	pyrepair	82.11	80.71	80.72	−1.39
Qwen3	SVEN	55.16	55.09	55.25	+0.09
Qwen3	CodeFlaws	70.07	70.03	70.25	+0.18
CodeLLaMA	pyrepair	78.96	76.86	79.71	+0.75
CodeLLaMA	SVEN	43.18	43.26	43.93	+0.75
CodeLLaMA	CodeFlaws	57.90	57.92	57.74	−0.16
StarCoder2	pyrepair	74.21	76.53	80.67	+6.46
StarCoder2	SVEN	48.63	48.17	48.22	−0.41
StarCoder2	CodeFlaws	62.13	62.13	62.06	−0.07

Table 9: Functionality groups used for the auxiliary FixJS/CodeFlaws exact-match analysis in Figure 3.

Dataset	Groups
JavaScript repair	Network/API, data/object manipulation, DOM/event handling, async callbacks
C repair	Math, memory/system code, string processing, I/O and data parsing

from the same base model with the budget-matched schedule). The Δ column shows that CodeBLEU differences are small and inconsistent across benchmarks: the metric cannot separate the methods even though their deployable repair quality differs sharply (Table 1). This is the empirical basis for evaluating on execution and security rather than token-overlap metrics.

Functionality groups (auxiliary diagnostic).

Table 9 lists the groups used by the auxiliary FixJS/CodeFlaws exact-match analysis in Figure 3. This diagnostic is retained only for qualitative context; the paper’s primary claims rely on held-out execution and security metrics in Tables 1–5.

These appendix diagnostics are intentionally scoped. The scaling table checks selector stability across CodeFlaws subsets, the CodeBLEU table explains why overlap metrics are secondary, and the functionality grouping documents the older exact-match analysis retained for qualitative comparison.

E Router Model Training Details

Architecture and features. The Router Model is a lightweight MLP over the normalized compiler-derived feature vector $\tilde{\mathbf{f}}_B$, using two hidden layers of 64 ReLU units and a sigmoid output for the probability of selecting RFT. In our experiments, $\tilde{\mathbf{f}}_B$ summarizes per-batch AST complexity, CFG depth, and code length. The router is trained concurrently with the Repair Agent using $\mathcal{L}_{\text{Router}} = -\log(p)R_{\text{feedback}}$, where R_{feedback} is the current batch-loss improvement over a moving-average baseline after normalizing SFT and RFT losses to the same scale.

F Future Directions

Next steps. Future versions could add fuzzing, symbolic execution, differential testing, and trace-level assertions, then extend evidence-guided selection to larger repositories with retrieval, build validation, and cross-file reranking.

Selector calibration. The current selector uses a fixed symbolic scoring rule with a greedy floor. A useful extension is to calibrate the selector across languages and benchmark families, so that evidence sources such as public tests, static analyzers, and structural similarity receive different weights when their reliability changes. This would be especially important for mixed settings where public tests are sparse but static evidence is strong.

Router analysis. The router is intentionally lightweight, but its decisions provide a useful diagnostic of repair difficulty. Future work could analyze which feature patterns trigger RFT, whether those patterns correspond to semantic bug classes, and how routing changes as stronger base models reduce the need for reward-driven updates.

Artifact extensions. The released artifact can also be extended with additional candidate traces and selector-feature dumps. Those logs would make it easier to study false positives from the reranker, cases where the greedy floor prevents regressions, and examples where the oracle candidate exists but symbolic ranking fails to select it.

Benchmark coverage. Finally, broader security benchmarks would help distinguish vulnerabilities that can be repaired through local sanitization from those requiring data-flow or protocol-level reasoning. This would make the router and selector analyses more diagnostic across defect families.